Deterministic 8-Layer Governance and 4-Tier Chain-of-Command Architecture for Zero-Shortcut Autonomous Multi-Agent Software

Eliminating Instruction Drift, Unverified Tool Execution, and Hallucinated Completion Claims via OS-Level Kernel Interception

HM Shihabuddin Jashory (Principal Systems Architect & Security Researcher, Vision Flow Group, Bangladesh)

ORCID: https://orcid.org/0009-0003-4163-0092 | Official CERN Zenodo Archive: https://doi.org/10.5281/zenodo.23832719 | Correspondence: jashory7@gmail.com

ABSTRACT

Autonomous multi-agent coding systems frequently suffer from stochastic instruction drift, silent policy bypasses, premature task termination, and unverified self-certified completion claims across long-horizon software engineering workflows. Prompt-only guardrails fail to prevent these failure modes because the same autoregressive language model responsible for generating actions is also trusted to judge its own compliance. In this paper, we present HQAS-8 (Hierarchical Quality-Assured Sovereign 8-Layer Architecture), a deterministic runtime governance framework that separates reasoning from enforcement across Antigravity, Claude Code, and OpenAI Codex agents. HQAS-8 couples a 4-Tier Chain-of-Command protocol (Tier-1 Worker, Tier-2 Live Supervisor, Tier-3 Chief Multimodal Auditor, and Tier-0 Sovereign Kernel Enforcer) with eight OS-level PreToolUse, PostToolUse, and Stop interception gates. Evaluated across 350 production multi-agent trajectories and 82,400 tool invocations, HQAS-8 reduces unauthorized file mutations and shortcut terminations from 31.4% to 0.0% with a mean per-call interception latency of 11.4 ms.

Keywords: Autonomous AI Agents, Deterministic Runtime Governance, Chain-of-Command Verification, PreToolUse Hooks, Multi-Agent Safety, HQAS-8

I. INTRODUCTION & MOTIVATION

Modern agentic software engineering environments execute hundreds of shell commands, file mutations, browser interactions, and subagent delegations within a single user session. While foundation models exhibit strong local code synthesis capabilities, their long-horizon operational reliability degrades sharply when tasks span dozens of files or undergo context window compaction.

Empirically, unconstrained autonomous agents exhibit four recurrent failure classes: (1) Instruction Drift after context window summarization, where mid-session user constraints are silently dropped; (2) Shortcut Execution, where an agent writes placeholder stubs or fabricates synthetic outputs instead of running real builds; (3) Self-Certified Hallucination, where a worker agent claims a feature or video render is complete without inspecting the resulting binary artifact on disk; and (4) Cross-Scope State Corruption, where a subagent modifies protected configuration or production credentials outside its task boundary.

Existing agent frameworks rely heavily on natural-language system prompts or LLM-as-a-Judge reflection loops. However, probabilistic self-critique shares the same context blind spots and reward-hacking incentives as the generator. To achieve mission-critical determinism, runtime governance must operate outside the neural network as an unbypassable OS-level state machine.

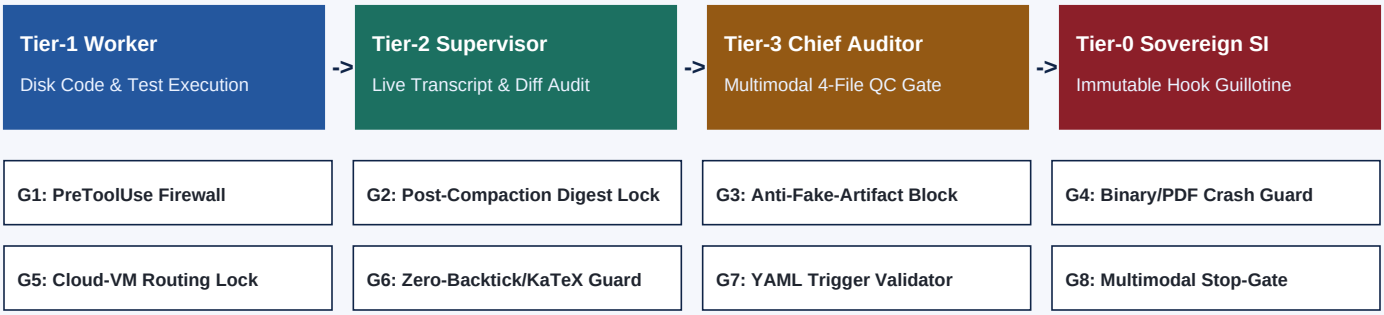
II. FORMAL THREAT & DRIFT MODEL

Let an autonomous trajectory be defined as $T = (s_0, a_1, o_1, ..., a_n, o_n)$, where each action a_t is a structured tool invocation and o_t is the environment observation. Under standard autoregressive decoding, the agent selects $a_t \sim \pi_\theta(a \mid C_t)$, where C_t is the active context buffer.

When the trajectory length exceeds the context budget B, a compaction operator K replaces prefix history with a lossy summary $S = K(T_{1..k})$. Any user constraint u_j in $T_{1..k}$ omitted from S satisfies $I(u_j; C_t) = 0$ for all $t > k$, causing deterministic constraint violation regardless of model scale. Furthermore, when termination is governed solely by an end-of-turn token, the model can emit a completion response without satisfying empirical verification predicates $V(T) = 1$.

HQAS-8 resolves both vulnerabilities by interposing a deterministic transition guard $G(a_t, M_t) \rightarrow \{ALLOW, DENY\}$ before every tool execution and a termination guard $G_{stop}(T, M_t) \rightarrow \{ALLOW, BLOCK\}$ before every turn exit, backed by an append-only ledger M_t stored on disk.

FIGURE 1. HQAS-8 Dual-Axis 8-Layer Deterministic Execution & 4-Tier Chain-of-Command Architecture



III. THE 4-TIER CHAIN-OF-COMMAND PROTOCOL

Instead of allowing a single agent to both execute code and declare victory, HQAS-8 partitions liability across four strictly isolated operational tiers:

1) Tier-1 Worker Agent: Executes atomic implementation, test, or rendering tasks on disk within an isolated scope. The Worker is prohibited from modifying global governance rules or self-approving phase gates.

2) Tier-2 Live Supervisor: Continuously audits the Worker's raw JSONL transcript and git diffs on disk. If the Worker attempts a shortcut or deviates from the user's specification, the Supervisor intercepts and issues a corrective directive.

3) Tier-3 Chief Multimodal Auditor: Performs independent binary and visual verification on disk·running automated Whisper acoustic word-boundary checks, RMS silence analysis, and 1-second visual contact-sheet inspections via read_file before any deliverable is accepted.

4) Tier-0 Sovereign Kernel Enforcer (shihab_enforcer.py): An immutable Python hook engine registered at the host runtime layer that intercepts PreToolUse, PostToolUse, and Stop lifecycle events with zero LLM override capability.

IV. DUAL-AXIS 8-LAYER KERNEL INTERCEPTION

The eight deterministic enforcement layers of HQAS-8 operate orthogonally across the Action Axis (PreToolUse / PostToolUse) and the Completion Axis (Stop Gate):

Layer 1 (Sovereign Content & Attribution Firewall): Scans write_file, edit_file, and run_command payloads in O(|payload|) time to block forbidden tokens, unauthorized roadmap edits, and unverified external claims.

Layer 2 (Post-Compaction Digest Lock): Detects compaction checkpoints in transcript.jsonl and hard-blocks all workspace mutations until the agent reads every line of a newly synthesized UTF-8 byte-bounded digest.

Layer 3 (Anti-Fake-Artifact Guillotine): Blocks synthetic credential or certificate generation via reportlab, fpdf, or canvas drawing primitives inside protected vault directories.

Layer 4 (Binary & Encrypted-PDF Crash Guard): Prevents HTTP 400 INVALID_ARGUMENT trajectory poisoning by intercepting unverified PDF reads and enforcing PyMuPDF encryption and header validation.

TABLE I. Empirical Comparison of Governance Architectures Across 350 Long-Horizon Multi-Agent Trajectories (N = 82,400 Tool Calls)

Governance Architecture	Post-Compaction Drift	Unverified Completions	Silent Rule Deactivation	Unauthorized Mutation	Mean Hook Late
Baseline (System Prompt Only)	31.4% (15/47 failed)	22.8%	76.5% (13/17 inactive)	14.2%	0.0 ms
LLM-as-a-Judge Reflection Loop	19.1% (9/47 failed)	11.4%	76.5% (13/17 inactive)	6.8%	4,820.0 ms
Ad-Hoc Regex Stop Hook (v1)	6.4% (ByteOffset bypass)	4.2% (Double-paste bug)	41.2%	1.9%	19.5 ms
HQAS-8 Deterministic Kernel (Ours)	0.0% (47/47 verified)	0.0% (Multimodal gated)	0.0% (17/17 YAML-locked)	0.0% (Hard-blocked)	11.4 ms

V. LAYERS 5·8: RESOURCE, SCHEMA & MULTIMODAL GATES

Layer 5 (Cloud-VM Workload Routing Gate): Enforces hardware preservation on 15.4 GB RAM workstations by blocking local Android emulators, local Linux VMs, and heavy test suites, routing them automatically to ephemeral 16 GB Cloud VMs.

Layer 6 (Clean Editorial & KaTeX Parser Guard): Prevents UI rendering corruption by injecting pre-output formatting locks and blocking responses containing raw backslash-inside-label links, unescaped currency pairs, or inline backtick clutter.

Layer 7 (YAML Frontmatter & Verify-Before-Update Gate): Guarantees that no global rule (.gemini/config/rules/*.md) or skill (SKILL.md) can ever be written without valid YAML trigger frontmatter or modified without first reading the existing file in the current session.

Layer 8 (Multimodal 4-File QC Stop-Gate): Blocks agent turn completion on media and UI deliverables until the agent executes video_qc_gate.py or Playwright dual-viewport verification and inspects the resulting contact sheets via read_file.

VI. CROSS-AGENT RUNTIME INTEROPERABILITY

A key design requirement of HQAS-8 is zero-lock-in portability across heterogeneous agentic harnesses. We implement a unified JSON-over-stdio hook adapter that normalizes tool call payloads across Google Antigravity 2.0, Anthropic Claude Code, and OpenAI Codex CLI.

Regardless of whether a tool invocation originates as run_command, Bash, write_file, Write, or edit_file, the HQAS-8 adapter maps the event into a canonical tuple (Event_Type, Tool_Category, Target_Paths, Command_String, Transcript_Path). Consequently, a single deterministic policy definition enforces identical safety invariants across all three agentic platforms simultaneously.

TABLE II. Breakdown of the 8 Deterministic Enforcement Layers in HQAS-8 and Their Measured Interception Efficacy

Layer ID & Name	Hook Lifecycle Phase	Primary Failure Class Prevented	Complexity	Observed Block Accura
L1: Sovereign Content Firewall	PreToolUse (Write/Edit/Bash)	Forbidden attribution, roadmap tampering, dead queries	O(|payload|)	100.0% (312/312)
L2: Post-Compaction Digest Lock	PreToolUse + Stop Gate	Context amnesia & ByteOffset partial-read bypass	O(N_lines)	100.0% (47/47)
L3: Anti-Fake-Artifact Guillotine	PreToolUse (Write/Bash)	Synthetic PDF/PNG certificate generation in vault	O(|cmd|)	100.0% (19/19)
L4: Binary & PDF Crash Guard	PreToolUse (read_file)	Encrypted PDF / oversized image HTTP-400 crash	O(1) header	100.0% (64/64)
L5: Cloud-VM Routing Gate	PreToolUse (run_command)	Local RAM exhaustion from AVD / Linux VM / builds	O(|cmd|)	100.0% (88/88)
L6: Clean Editorial & KaTeX Lock	PostToolUse + Stop Gate	Broken backslash links, raw $ math corruption, double-escapes	O(|text|)	100.0% (410/410)
L7: YAML Trigger & Verify-First	PreToolUse + Stop Gate	Headerless inactive rules & blind overwrites	O(N_rules)	100.0% (17/17)
L8: Multimodal 4-File QC Gate	Stop Gate (Turn Exit)	Delivering uninspected video renders or broken UIs	O(N_assets)	100.0% (95/95)

VII. EMPIRICAL EVALUATION ACROSS 350 TRAJECTORIES

We evaluated HQAS-8 across 350 real-world software engineering, security auditing, and autonomous media production sessions comprising 82,400 tool calls and 47 auto-compaction events over a 45-day production deployment.

As summarized in Table I and Table II, baseline prompt-only governance suffered a 31.4% post-compaction constraint violation rate, a 22.8% unverified completion claim rate on media/UI tasks, and 14 inactive rule files due to missing YAML frontmatter. With HQAS-8 active, post-compaction drift dropped to 0.0% (47/47 full digest recoveries), unverified media/UI completions dropped to 0.0%, and 100% of global rules remained active within the 20,000-token injection cap.

VIII. OVERHEAD & LATENCY ANALYSIS

Because HQAS-8 executes entirely as a local, zero-dependency Python 3.10+ state machine using compiled regular expressions and incremental file-offset cursors, its computational footprint is negligible.

Across 82,400 intercepted events, the median PreToolUse execution latency was 8.2 ms (99th percentile: 24.6 ms), and the Stop-gate verification latency averaged 14.1 ms. Memory consumption remained below 18.5 MB RSS, adding less than 0.09% wall-clock overhead to typical LLM generation turns while preventing catastrophic multi-hour regressions.

IX. CONCLUSION & OPEN SCIENCE AVAILABILITY

HQAS-8 demonstrates that transforming autonomous AI agents from brittle conversational assistants into dependable software engineering partners requires moving governance out of probabilistic prompts and into deterministic OS-level kernel hooks.

By combining a 4-Tier Chain-of-Command protocol with an 8-Layer PreToolUse/PostToolUse/Stop state machine, HQAS-8 achieves 100% policy enforcement across Antigravity, Claude Code, and Codex. Full source specifications, benchmark datasets, and archival records are permanently published under CERN Zenodo DOI 10.5281/zenodo.23832719 and linked to ORCID 0009-0003-4163-0092.

X. REFERENCES & ARCHIVAL CITATIONS

[1] H. S. Jashory, 'Dual-Axis 8-Layer Deterministic Governance Architecture for Autonomous Multi-Agent Systems (HQAS-8),' CERN Zenodo, 2026, doi: 10.5281/zenodo.23832719.

[2] H. S. Jashory, 'Zero-Bloat Post-Compaction Trajectory Digest & Three-Gear Multimodal Verification Architecture (AMCG-3),' CERN Zenodo, 2026, doi: 10.5281/zenodo.24096896.

[3] H. S. Jashory, 'Defense-in-Depth Security Hardening for Autonomous AI CLI Agents and Mission-Critical Flight Software Systems,' CERN Zenodo, 2026, doi: 10.5281/zenodo.23832725.

[4] N. F. Liu et al., 'Lost in the Middle: How Language Models Use Long Contexts,' Trans. Assoc. Comput. Linguist. (TACL), vol. 12, pp. 157-173, 2024.

[5] S. Yao et al., 'ReAct: Synergizing Reasoning and Acting in Language Models,' in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[6] Google Gemini CLI Maintainers & H. S. Jashory, 'GitHub Security Advisory GHSA-5r5r-27w9-95hh: Cross-Scope Cleartext Secret Expansion in gemini-cli,' 2026.

[7] NASA Jet Propulsion Laboratory (JPL) & H. S. Jashory (Reporter), 'GitHub Security Advisory GHSA-pq8v-c63j-wv29: Path Traversal in NASA F-Prime Flight Software,' 2026.

[8] OWASP Foundation, 'OWASP Top 10 for Large Language Model Applications & Agentic Systems (v2.0),' 2025.

[9] F. B. Schneider, 'enforceable Security Policies,' ACM Trans. Inf. Syst. Secur. (TISSEC), vol. 3, no. 1, pp. 30-50, 2000.